

DBXKit简介

2024-08-05 11:08

目录

1. Chain

1.1 库简介

1.2 使用样例

2. Tools工具

2.1 DBXTaskQueueManager

2.2 DBXTaskTimerManager

3. Views自定义视图

3.1 MnaLabelsView

3.2 使用样例

4. Extension扩展类

4.1 解析后台数据NSDictionary+dbx_valuePath

4.2 扩大视图点击区域UIView+dbx_clickedArea

4.3 监听对象属性值NSObject+PropertyObserver

4.4 富文本NSAttributedString+dbx

4.5 NSObject+dbx_modelValue

5. Debounce 函数防抖

6. Syringe 依赖注入库

DBXKit项目开发过程中累计的常用工具库，是对其他第三方库的补充，按功能分为多个子库，core是整个DBXKit的公共库，功能用于内部使用，无需关注，主要用到的是以下子库

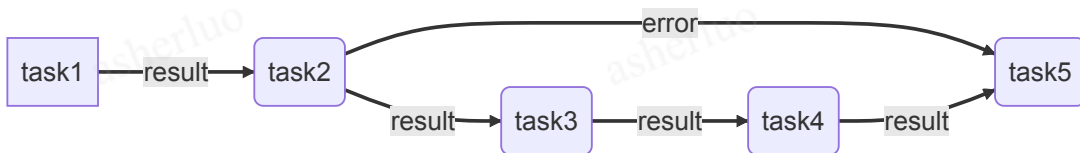
1. Chain

1.1 库简介

任务链式执行管理库，目前加速器里我用的比较多，可以简化逻辑，特点：

- 链条式调用
- 一个任务是一个DBXChainTask实例，每个任务按照链接顺序执行，可以很方便的修改需求，包括任务的增减改，只需要改一下链接方式即可
- 支持同步和异步任务
- 任务执行成功，就给task.result赋值，失败给task.error赋值，这两个任何一个赋值即表示当前task执行完毕，进入下一个任务

每个task可以是同步或者异步，task完成后会自动进入下一个task，并且会把前一个task的结果带到下一个task，如下图：



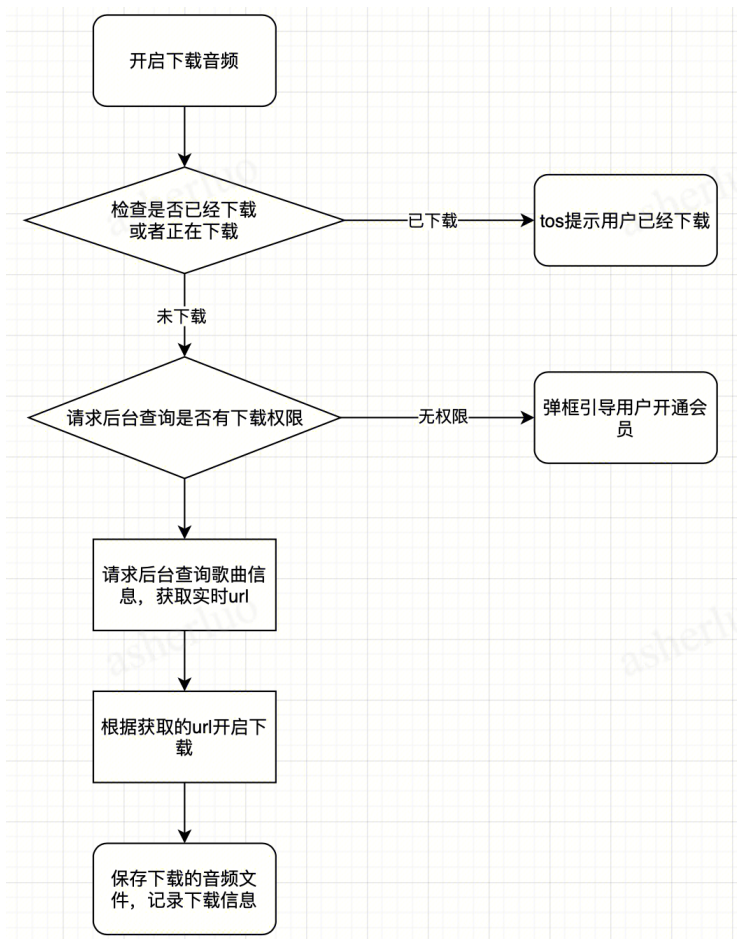
```

1 // 模拟4个异步请求任务111 222 333 444，按顺序依次执行，每个请求花费的时间假定为2秒
2 - (IBAction)testChainTask {
3     dispatch_queue_t queue = dispatch_queue_create("asherluo", nil);
4     [[[[self createTaskWithName:@"111"] thenWithBlock:^(id
        _Nullable(DBXChainTask * _Nonnull task) {
5         DBXChainTask *next = [self createTaskWithName:@"222"];
6         return next;
7     } ] thenWithBlock:^(id _Nullable(DBXChainTask * _Nonnull task) {
8         if (task.error) {
9             return task.error;
10        }
11        DBXChainTask *next = [self createTaskWithName:@"333"];
12        return next;
13    } ] thenWithBlock:^(id _Nullable(DBXChainTask * _Nonnull task) {
14        DBXChainTask *next = [self createTaskWithName:@"444" sleep:0];
15        return next;
16    }]];
17 }
18
19 // 创建一个task任务，命名为name
20 - (DBXChainTask *)createTaskWithName:(NSString *)name {
21     DBXChainTask *task = [DBXChainTask chainTask];
22     task.taskName = name;
23     dispatch_after(dispatch_time(DISPATCH_TIME_NOW, (int64_t)(2 *
        NSEC_PER_SEC)), dispatch_get_main_queue(), ^{
24         if (succ) {
25             [task setResult:@{@"res":@"succ"}];
26         } else {
27             [task setError:[NSError errorWithDomain:[NSString
                stringWithFormat:@"%@" failed",task.taskName ] code:-1 userInfo:nil]];
28         }
29     });
30     return task;
31 }

```

1.2 使用样例

如语音包下载的需求用Chain来完成
需求流程图



代码调用，对应上面流程图

```

- (void)downloadMusic:(NSString *)mid cdTime:(NSInteger)cd complete:(void (^)(id data))complete {
    __weak __typeof(self) weakSelf = self;
    [[[[self confirmShouldDownload:mid cdTime:cd] thenWithBlock:^(id _Nullable(DBXChainTask * _Nonnull) task) {
        if (task.error) {
            return task.error;
        }
        return [self checkUserPermission:@"download_song"];
    }] thenWithBlock:^(id _Nullable(DBXChainTask * _Nonnull) task) {
        if (task.error) {
            return task.error;
        }
        [[BaseBeaconReporter sharedInstance] reportMessagePvType:@"biayin_download_song" param:mid];
        [QMUITips showWithText:@"歌曲开始下载" detailText:@"可在「我的声音」里查看下载进度" inView:self.view];
        return [self requestMusicInfo:mid];
    }] thenWithBlock:^(id _Nullable(DBXChainTask * _Nonnull) task) {
        if (task.error) {
            return task.error;
        }
        MnaMusicModel *model = task.result;
        [weakSelf.downloadDB saveDownloadMusic:model];
        MnaMusicDownloadPathModel *pathModel = [MnaMusicDownloadPathModel downloadPathWith:model.material_id path:nil downloadRate:0 source:@"download"];
        [weakSelf.downloadDB saveMusicFilePath:pathModel];
        return [MnaMusicTools saveMusicToPhoto:model progress:^(MnaMusicModel * _Nonnull downloadModel, float progress) {
            if ([weakSelf.delegate respondsToSelector:@selector(musicBarController:musicModel:downloadProgress:)] {
                dispatch_async(dispatch_get_main_queue(), ^{
                    [weakSelf.delegate musicBarController:weakSelf musicModel:downloadModel downloadProgress:progress];
                });
            }
        }];
    }]);
}

// 110是用户取消重复下载
if (task.error.code == -110) {
    return nil;
}

if ([weakSelf.delegate respondsToSelector:@selector(musicBarController:musicID:downloadFinish:)]) {
    [weakSelf.delegate musicBarController:weakSelf musicID:mid downloadFinish:task.error];
}

// kDBXChainMultipleErrorsCode是用户暂停当前正在执行的下载任务
if (task.error.code == kDBXChainMultipleErrorsCode) {
    return nil;
}

if (task.error) {
    if ([task.error.localizedRecoverySuggestion isEqualToString:@"openvip"]) {
        [[MnaMediator sharedInstance] mnaMediator_showCommonDialogue:@{
            @"img" : [UIImage mna_plazaImageNamed:@"icon_music_download_vip"],
            @"action" : @"mna://Payment/gotoPayment"
        }];
    } else {
        [QMUITips showError:task.error.localizedDescription inView:weakSelf.view];
    }
} else if (task.result) {
    NSURL *url = task.result;
    MnaMusicDownloadPathModel *pathModel = [MnaMusicDownloadPathModel downloadPathWith:mid path:url.path downloadRate:1.0 source:@"download"];
    [weakSelf.downloadDB saveMusicFilePath:pathModel];
}

if (complete) {
    complete(@{@"errCode":@(task.error.code), @"msg":task.error.localizedDescription?:@"请求出错"});
}

return nil;
}
}

```

2. Tools工具

2.1 DBXTaskQueueManager

串行执行队列，任务添加进队列，启动队列后按添加顺序执行任务

2.2 DBXTaskTimerManager

定时器任务，设置循环执行任务

3. Views自定义视图

3.1 MnaLabelsView

- 多个标签视图
- 支持纯文字、纯图片、图文混编
- 纯文字风格只需要传入文字内容，会根据文字长度计算设置标签长度，支持一定程度的自定义

3.2 使用样例

如下图种红色标出的样式

固定长度的纯文本



图文混编



可变长度的纯文本



纯图片



纯图片



4. Extension扩展类

4.1 解析后台数据NSDictionary+dbx_valuePath

从后台获取数据后解析其中字段，代码比较简单，但是很实用，我个人用的比较多，如后台数据格式：


```

1  {
2      content : {
3          material : {
4              material_id : xxx,
5              name : xxx,
6              meterial_status : 1
7          }
8      }
9  }

```

要解析出material数据，调用方式如下，无需添加过多的数据安全判定

```

1  NSDictionary *materDic = [dictionary dbx_valueForKeyPath:@"content.material"
    limitedClass:[NSDictionary class]];

```

支持同时解析多个字段

如上要解析name和meterial_status，调用方式如下：

```

1  // 传入指针后需要的数据解析后会传入指针中，可以同时解析任意数量的字段
2  NSString *name;
3  NSNumber *status;
4  [dictionary dbx_valuesForKeyPath:@"content.material.name&meterial_status"
    values:&name, &status, nil];

```

4.2 扩大视图点击区域UIView+dbx_clickedArea

扩大视图的可点击（hittest）区域，可以搭配MnaDebuggingOnLine使用，MnaDebuggingOnLine已支持此接口的可视化

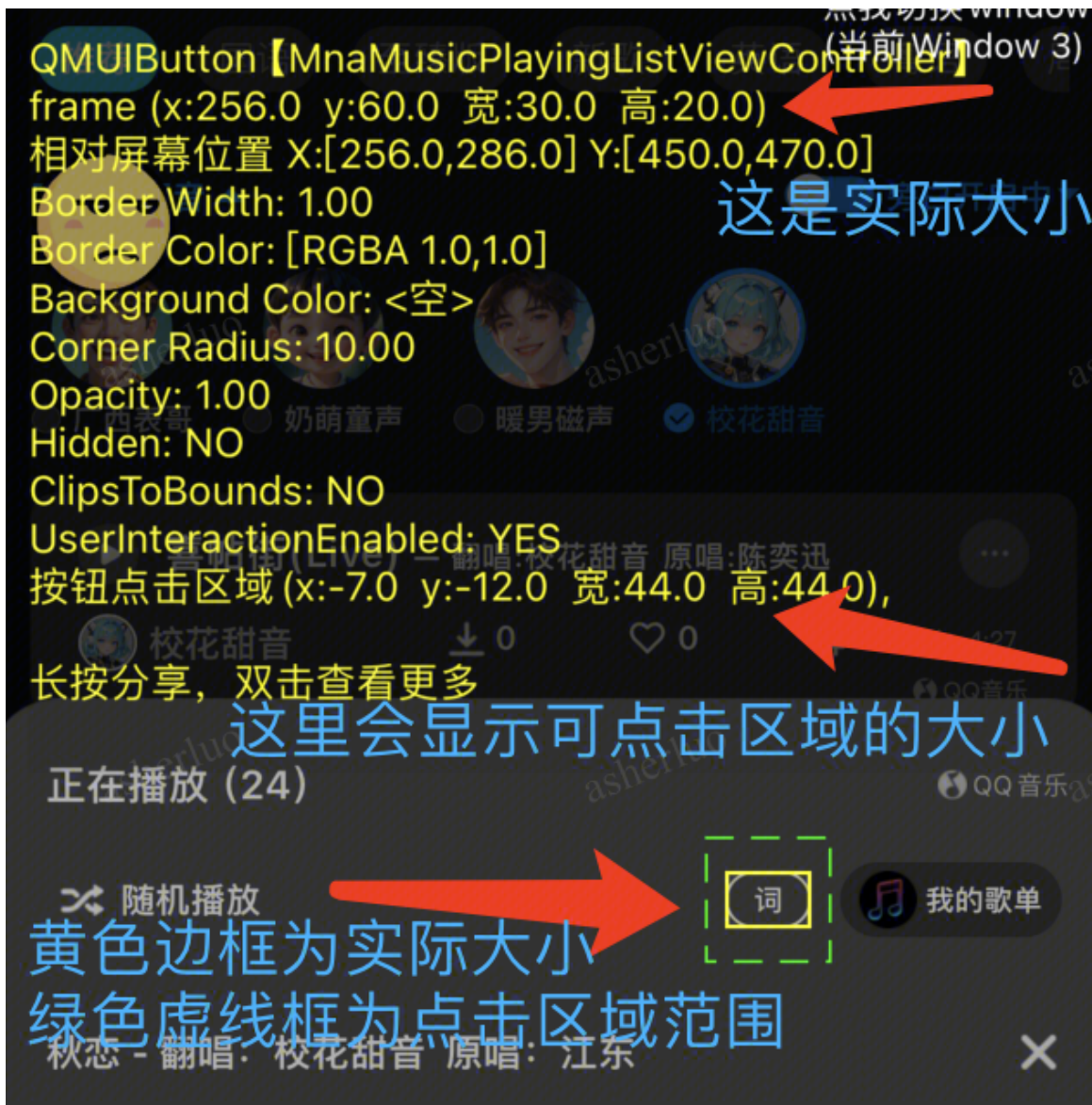
主要有2个接口：

```

1  // 把视图的hittest宽高以自身中心往外扩展，不够44的补齐到44，超过44的保留自身大小
2  - (void)dbx_enableExtendedClickedAreaFix44x44;

```

如下图，歌词按钮设计稿上尺寸是宽30，高20，达不到交互规范要求的44，因此使用dbx_enableExtendedClickedAreaFix44x44来扩大点击区域，通过MnaDebuggingOnLine组件查看扩大后的情况，如下图



```

1 // 自定义设定需要扩展的方向
2 - (void)dbx_enableExtendedClickedAreaEdgeInsets:
    (UIEdgeInsets)clickedAreaEdgeInsets
  
```

有些情况无法直接往外规则的扩展，可以用上面接口修改扩展的方向，如下图绿色虚线区域，上面空间不够，只能延伸一点点，底部空间够可以多多延伸一些



4.3 监听对象属性值NSObject+PropertyObserver

监听实例对象的属性值的变化，使用上类似RAC，但是很轻量，就简单的KVO运用

```

1 // 监听ViewModel实例的属性voiceTypeList, 当voiceTypeList值发生变化时更新页面
2 __weak __typeof(self)weakSelf = self;
3 [self.viewModel dbx_addObserverForKeyPath:@"voiceTypeList" valueChange:^(id
  _Nonnull value) {
4     if (![value isKindOfClass:[NSArray class]]) {
5         return;
6     }
7     [weakSelf.styleCollectionView reloadDataWithList:value];
8 }];

```

4.4 富文本NSAttributedString+dbx

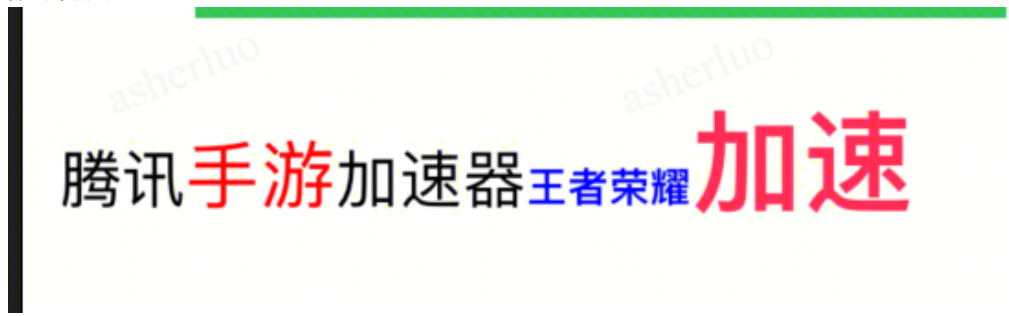
可以方便的给一段文本的任意位置设置富文本属性，如下示例

```

1 // 初始化一个富文本，标记a,b,c3个需要设置属性的位置
2 NSAttributedString *attStr = [[NSAttributedString alloc] initWithString:
  [NSString stringWithFormat:@"腾讯%@手游%@加速器%@王者荣耀%@@加速%@",
  @"a".beginDelimiter, @"a".endDelimiter, @"b".beginDelimiter,
  @"b".endDelimiter, @"c".beginDelimiter, @"c".endDelimiter]];
3
4 // 对标记a的位置赋值
5 attStr = [attStr
  dbx_addAttributes:@{NSForegroundColorAttributeName:UIColor.redColor,NSFontAttri
  [UIFont systemFontOfSize:20]} delimiter:@"a"];
6
7 // 对标记b的位置赋值
8 attStr = [attStr
  dbx_addAttributes:@{NSForegroundColorAttributeName:UIColor.blueColor,NSFontAttri
  [UIFont boldSystemFontOfSize:11]} delimiter:@"b"];
9
10 // 对标记c的位置赋值
11 attStr = [attStr
  dbx_addAttributes:@{NSForegroundColorAttributeName:UIColor.systemPinkColor,NSFo
  [UIFont boldSystemFontOfSize:30]} delimiter:@"c"];

```

效果如图



腾讯手游加速器王者荣耀加速

4.5 NSObject+dbx_modelValue

目前的数据解析都是NSDictionary和数据模型Model之间转换，生成新的Model，这个库用于给已经初始化了的数据模型Model的属性赋值

5. Debounce 函数防抖

可以用来做什么：

- 在不改动方法（如methodA）的实现的情况下，给methodA设置调用频率，比如2秒内最多执行1次，无论调用方法调用了多少次

```
1  - (void)methodA {
2      print("methodA call");
3  }
4  while (count < 100) {
5      [self methodA];
6  }
7  打印结果：methodA call
```

- 禁止方法执行

```
1  //禁止参数传入nil
2  - (void)methodB:(NSString *)a {
3      print(a);
4  }
5  [self methodB:@"abc"];
6  [self methodB:nil];
7  [self methodB:@"efg"];
8  // 通过组件设定不能传入nil后
9  打印结果：
10 abc
11 efg
```

- 修改方法的参数和返回值

```
1    //
2    - (NSString *)methodC:(NSString *)a {
3        print("hello %@",a);
4        return @"I'm andy";
5    }
6    NSString *result = [self methodC:@"lili"];
7    print(result);
8    // 通过组件把参数修改为joy,返回值修改为I'm asher
9    打印结果：
10   hello joy
11   I'm asher
```

限频支持3种模式

- first : 规定的时间内（如2秒内）只执行第一次，后面的丢掉
- last : 规定的时间内（如2秒内）只执行最后一次，前面的丢掉
- debounce : 规定的时间内（如2秒内）发送消息后延迟一段时间执行，如果继续发送消息，会重新计时

6. Syringe 依赖注入库

依赖注入组件，编写实例的描述代码，根据描述代码生成实例